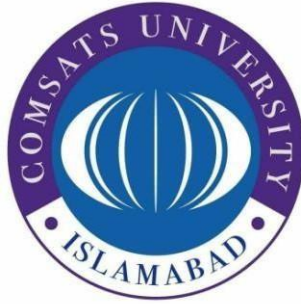


COMSATS University Islamabad

Attock Campus



Department Of Computer Science

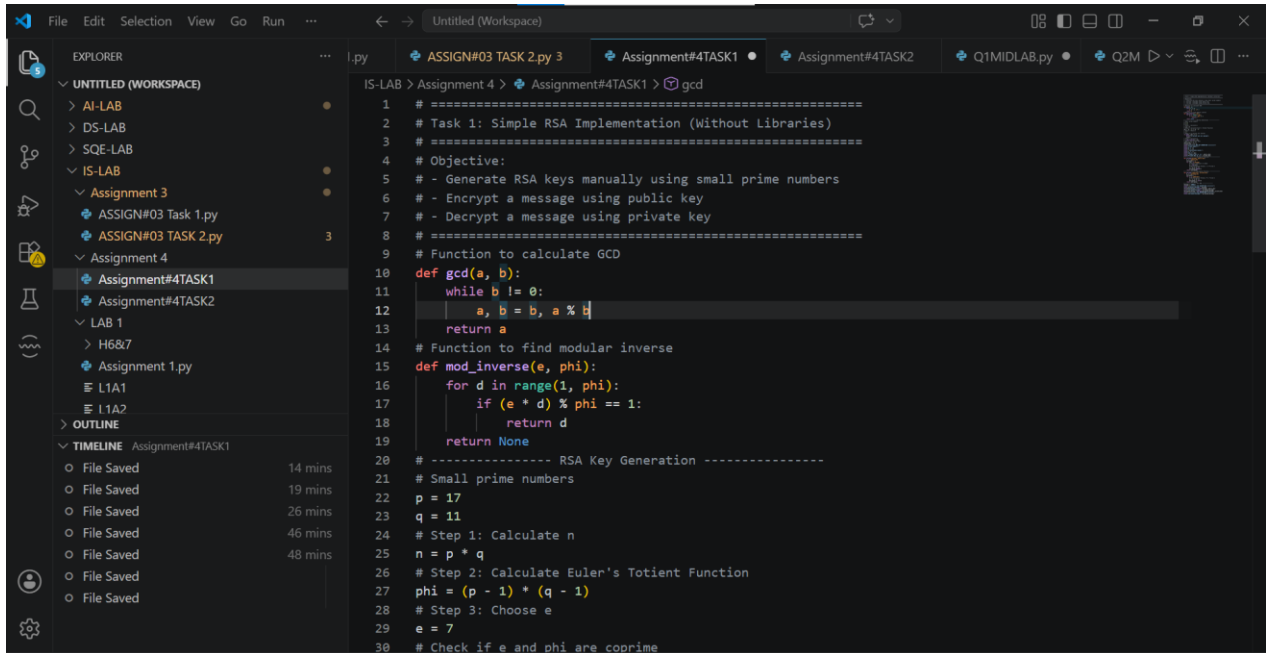
<i>Course</i>	<i>IS-LAB</i>
<i>Instructor</i>	<i>Ms. AMBREEN</i>
<i>Assignment No.</i>	<i>04</i>

Student Details

<i>Registration No.</i>	<i>Name</i>
<i>FA24-BSE-024</i>	<i>ISMAIL AHMED</i>

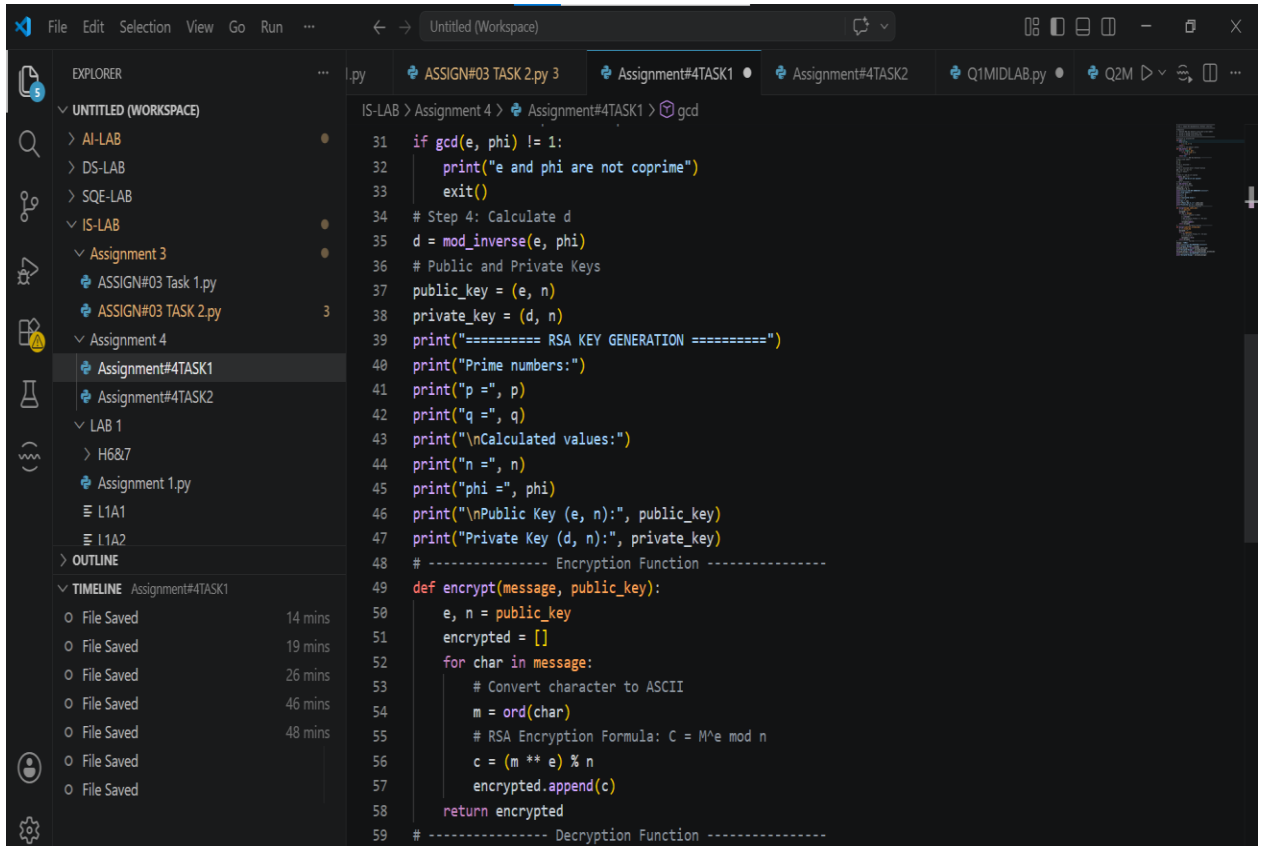
Task 1: Simple RSA Implementation (Without Libraries):

CODE:



This screenshot shows the first part of the RSA implementation code in a VS Code editor. The Explorer panel on the left shows the project structure with files like 'Assignment#03 Task 1.py', 'Assignment#03 TASK 2.py', and 'Assignment#4TASK1'. The main editor window displays the following code:

```
1 # =====  
2 # Task 1: Simple RSA Implementation (Without Libraries)  
3 # =====  
4 # Objective:  
5 # - Generate RSA keys manually using small prime numbers  
6 # - Encrypt a message using public key  
7 # - Decrypt a message using private key  
8 # =====  
9 # Function to calculate GCD  
10 def gcd(a, b):  
11     while b != 0:  
12         a, b = b, a % b  
13     return a  
14 # Function to find modular inverse  
15 def mod_inverse(e, phi):  
16     for d in range(1, phi):  
17         if (e * d) % phi == 1:  
18             return d  
19     return None  
20 # ----- RSA Key Generation -----  
21 # Small prime numbers  
22 p = 17  
23 q = 11  
24 # Step 1: Calculate n  
25 n = p * q  
26 # Step 2: Calculate Euler's Totient Function  
27 phi = (p - 1) * (q - 1)  
28 # Step 3: Choose e  
29 e = 7  
30 # Check if e and phi are coprime
```



This screenshot shows the second part of the RSA implementation code in a VS Code editor. The Explorer panel on the left shows the project structure with files like 'Assignment#03 Task 1.py', 'Assignment#03 TASK 2.py', and 'Assignment#4TASK1'. The main editor window displays the following code:

```
31 if gcd(e, phi) != 1:  
32     print("e and phi are not coprime")  
33     exit()  
34 # Step 4: Calculate d  
35 d = mod_inverse(e, phi)  
36 # Public and Private Keys  
37 public_key = (e, n)  
38 private_key = (d, n)  
39 print("===== RSA KEY GENERATION =====")  
40 print("Prime numbers:")  
41 print("p =", p)  
42 print("q =", q)  
43 print("\nCalculated values:")  
44 print("n =", n)  
45 print("phi =", phi)  
46 print("\nPublic Key (e, n):", public_key)  
47 print("Private Key (d, n):", private_key)  
48 # ----- Encryption Function -----  
49 def encrypt(message, public_key):  
50     e, n = public_key  
51     encrypted = []  
52     for char in message:  
53         # Convert character to ASCII  
54         m = ord(char)  
55         # RSA Encryption Formula: C = M^e mod n  
56         c = (m ** e) % n  
57         encrypted.append(c)  
58     return encrypted  
59 # ----- Decryption Function -----
```

```

60 def decrypt(ciphertext, private_key):
61     d, n = private_key
62     decrypted = ""
63     for c in ciphertext:
64         # RSA Decryption Formula: M = C^d mod n
65         m = (c ** d) % n
66         decrypted += chr(m)
67     return decrypted
68 # ----- Test RSA -----
69 message = "ISMAIL"
70 print("\n===== RSA ENCRYPTION =====")
71 print("Original Message:", message)
72 encrypted_message = encrypt(message, public_key)
73 print("Encrypted Message:", encrypted_message)
74 decrypted_message = decrypt(encrypted_message, private_key)
75 print("\n===== RSA DECRYPTION =====")
76 print("Decrypted Message:", decrypted_message)

```

OUTPUT:

The screenshot shows a Python IDE with a terminal window. The terminal output is as follows:

```

PS D:\IS-LAB> & "C:/Users/ISMAIL AHMED/AppData/Local/Microsoft/WindowsApps/python3.13.exe" "d:/IS-LAB/Assignment 4/Assignment#4TASK1"

Calculated values:
n = 187
phi = 160

Public Key (e, n): (7, 187)
Private Key (d, n): (23, 187)

===== RSA ENCRYPTION =====
Original Message: ISMAIL
Encrypted Message: [61, 8, 121, 142, 61, 32]

===== RSA DECRYPTION =====
Decrypted Message: ISMAIL
PS D:\IS-LAB>

```

LINE BY LINE EXPLANATION:

1. Helper Functions

These functions handle the mathematical logic required for RSA.

- **def gcd(a, b)::** Defines a function to find the Greatest Common Divisor using the Euclidean Algorithm.
- **while b != 0::** A loop that continues until the remainder is zero.
- **a, b = b, a % b:** Updates a to the previous divisor and b to the remainder of a/b.
- **return a:** Returns the GCD; if it is 1, the numbers are "coprime."

- **def mod_inverse(e, phi)::** Defines a function to find the modular multiplicative inverse of $e \pmod{\phi}$.
- **for d in range(1, phi)::** Checks every number from 1 up to the value of ϕ .
- **if (e * d) % phi == 1::** Checks the condition $e \cdot d \equiv 1 \pmod{\phi}$.
- **return d:** Returns the private exponent d once the condition is met.

2. RSA Key Generation

This section generates the public and private keys using number theory.

- **p = 17, q = 11:** Two small prime numbers chosen as the foundation of the keys.
- **n = p * q:** Calculates the modulus n (187), which is part of both keys.
- **phi = (p - 1) * (q - 1):** Calculates Euler's Totient Function, $\phi(n)$, which determines the "space" for exponents.
- **e = 7:** Chooses the public exponent e . It must be coprime with ϕ .
- **if gcd(e, phi) != 1::** Validates that e and ϕ do not share factors.
- **d = mod_inverse(e, phi):** Computes the private exponent d , which "undoes" the encryption.
- **public_key = (e, n):** Packages e and n together for encryption.
- **private_key = (d, n):** Packages d and n together for decryption.

3. Encryption Function

This converts plain text into a scrambled list of numbers.

- **def encrypt(message, public_key)::** Defines the encryption process using the public key.
- **e, n = public_key:** Unpacks the values e and n from the tuple.
- **encrypted = []:** Initializes an empty list to store the resulting ciphertext values.
- **for char in message::** Iterates through each character in the string "ISMAIL".

- **m = ord(char):** Converts the character into its numerical ASCII equivalent (e.g., 'I' becomes 73).
- **c = (m e) % n:** Applies the RSA formula: $C = M^e \pmod{n}$.
- **encrypted.append(c):** Adds the encrypted number to our list.

4. Decryption Function

This converts the scrambled numbers back into the original text.

- **def decrypt(ciphertext, private_key):** Defines the decryption process using the private key.
- **d, n = private_key:** Unpacks the values d and n .
- **decrypted = "":** Initializes an empty string to build the final message.
- **for c in ciphertext:** Iterates through each encrypted number in the list.
- **m = (c d) % n:** Applies the RSA decryption formula: $M = C^d \pmod{n}$.
- **decrypted += chr(m):** Converts the resulting number back into its ASCII character and appends it to the string.

5. Execution and Testing

- **message = "ISMAIL":** The original text to be protected.
- **encrypted_message = encrypt(...):** Calls the function to produce the ciphertext.
- **decrypted_message = decrypt(...):** Calls the function to recover the original message.
- **print(...):** Displays the keys, the ciphertext (numbers), and the final decrypted string to verify the process worked.

SECURITY ANALYSIS:

1. Small Prime Numbers (Key Size)

- **The Issue:** The code uses $p=17$ and $q=11$, resulting in $n=187$.
- **The Risk:** A modern computer can factor $n=187$ into its primes in a fraction of a microsecond. Once p and q are known, an attacker can calculate ϕ and then d (the private key) effortlessly.

- **The Standard:** Real-world RSA typically uses primes that result in an n of **2048 or 4096 bits** in length.

2. Lack of Padding (Deterministic Encryption)

- **The Issue:** This implementation is "Electronic Codebook" (ECB) style—it encrypts each character individually and directly.
- **The Risk:** Because the encryption is deterministic, the same letter will always result in the same number. For example, in the name "ISMAIL", the letter 'I' appears twice; it will produce the same encrypted value both times. An attacker can use **frequency analysis** or a **replay attack** to guess the message without ever breaking the encryption key.
- **The Standard:** Modern RSA uses **OAEP (Optimal Asymmetric Encryption Padding)**, which adds random data to the message before encryption, ensuring that encrypting the same word twice results in two completely different ciphertexts.

3. Use of $\text{pow}(m, e) \% n$ vs. $\text{pow}(m, e, n)$

- **The Issue:** The code calculates $(m^e) \% n$.
- **The Risk:**
 1. **Efficiency:** For large exponents, m^e creates a massive intermediate number that can crash memory.
 2. **Side-Channel Attacks:** Simple power functions take different amounts of time to calculate depending on the bits of the key. Attackers can measure the time your CPU takes to process a request to "leak" the private key.
- **The Standard:** Python's built-in $\text{pow}(m, e, n)$ function is much faster and more resistant to certain timing attacks because it uses "Modular Exponentiation."

4. Integer-to-Character Mapping

- **The Issue:** The code converts the message character-by-character.
- **The Risk:** RSA is not meant to encrypt long strings of text directly because it is mathematically heavy.

- **The Standard:** In practice, RSA is used for **Hybrid Encryption**. RSA encrypts a small "Symmetric Key" (like AES), and then that AES key is used to encrypt the actual message. This combines the security of RSA with the speed of AES.

5. Vulnerability to Brute Force

- **The Issue:** The `mod_inverse` function uses a simple for loop.
- **The Risk:** This is incredibly slow for large numbers. While fine for $n=187$, it would take millions of years to find a private key for a 2048-bit n .
- **The Standard:** Professionals use the **Extended Euclidean Algorithm**, which finds the modular inverse almost instantly, even for massive numbers.

Task 2: Create a Digital Signature:

CODE:

```

1  # =====
2  # Digital Signature using RSA + SHA256
3  # =====
4  import hashlib
5  # ----- RSA KEY GENERATION -----
6  def gcd(a, b):
7      while b != 0:
8          a, b = b, a % b
9      return a
10 def mod_inverse(e, phi):
11     for d in range(1, phi):
12         if (e * d) % phi == 1:
13             return d
14     return None
15 # Small prime numbers
16 p = 61
17 q = 53
18 n = p * q
19 phi = (p - 1) * (q - 1)
20 e = 17 # public exponent
21 d = mod_inverse(e, phi)
22 public_key = (e, n)
23 private_key = (d, n)
24 print("Public Key:", public_key)
25 print("Private Key:", private_key)
26 # ----- HASH FUNCTION -----
27 def create_hash(message):
28     return hashlib.sha256(message.encode()).hexdigest()
29 # ----- SIGN MESSAGE -----
30 def sign_message(message, private_key):

```

The screenshot shows a VS Code editor with a Python file named 'Assignment#4TASK2.py'. The code implements RSA signing and verification. It includes functions for signing a message and verifying a signature. The main execution part demonstrates signing the message 'Hello RSA' and then verifying it. The code is as follows:

```
30 def sign_message(message, private_key):
31     d, n = private_key
32     # Step 1: Hash the message
33     message_hash = create_hash(message)
34     # Step 2: Convert hash to integer
35     hash_int = int(message_hash, 16)
36     # Step 3: Sign using private key
37     signature = pow(hash_int, d, n)
38     return signature
39 # ----- VERIFY SIGNATURE -----
40 def verify_signature(message, signature, public_key):
41     e, n = public_key
42     # Hash the message again
43     new_hash = create_hash(message)
44     new_hash_int = int(new_hash, 16)
45     # Decrypt signature using public key
46     decrypted_hash = pow(signature, e, n)
47     return (new_hash_int % n) == decrypted_hash
48 # ----- TEST CASES -----
49 message = "Hello RSA"
50 print("\nOriginal Message:", message)
51 # Sign
52 signature = sign_message(message, private_key)
53 print("Signature:", signature)
54 # Verify original
55 result1 = verify_signature(message, signature, public_key)
56 print("Verification (Original):", result1)
57 # Modify message
58 modified_message = "Hello rsa"
59 print("\nModified Message:", modified_message)
60 # Verify modified
61 result2 = verify_signature(modified_message, signature, public_key)
62 print("Verification (Modified):", result2)
```

OUTPUT:

The screenshot shows the terminal output of the program. The output is as follows:

```
PS D:\IS-LAB> "C:/Users/ISMAIL AHMED/AppData/Local/Microsoft/WindowsApps/python3.13.exe" "d:/IS-LAB/Assignment 4/Assignment#4TASK2.py"
Public Key: (17, 3233)
Private Key: (2753, 3233)

Original Message: Hello RSA
Signature: 1027
Verification (Original): True

Modified Message: Hello rsa
Verification (Modified): False
PS D:\IS-LAB>
```

LINE BY LINE EXPLANATION:

1. Setup and Key Generation

- **import hashlib:** Imports Python's library for secure hashing (SHA-256).

- **p = 61, q = 53**: Slightly larger primes than before, providing $n = 3233$ and $\phi = 3120$.
- **e = 17**: The public exponent.
- **d = mod_inverse(e, phi)**: Calculates the private key used for "signing."
- **public_key, private_key**: Tuples storing the keys.

2. The Hash Function

- **def create_hash(message)::** RSA is too slow to sign large files directly, so we sign a "fingerprint" of the data instead.
- **hashlib.sha256(message.encode()).hexdigest()**: Takes the text, encodes it to bytes, runs the SHA-256 algorithm, and returns a 64-character hexadecimal string.

3. Signing the Message (Using Private Key)

- **message_hash = create_hash(message)**: Creates the unique fingerprint of the message.
- **hash_int = int(message_hash, 16)**: Converts the hexadecimal hash into a massive integer so math can be performed on it.
- **signature = pow(hash_int, d, n)**: The Signing Step. It applies the private key d to the hash. This proves the sender is the only one who could have created this value.

4. Verifying the Signature (Using Public Key)

- **new_hash = create_hash(message):** The receiver hashes the message they received.
- **decrypted_hash = pow(signature, e, n):** The receiver "unlocks" the signature using the sender's public key e .
- **return (new_hash_int % n) == decrypted_hash:** If the hash the receiver calculated matches the hash stored in the signature, the message is authentic.

Security Analysis:

1. The "Hash > Modulus" Problem (Critical Vulnerability)

- **The Issue:** A SHA-256 hash is 256 bits long. However, $n = 3233$ is only about 12 bits long.
- **The Risk:** In the line `new_hash_int % n`, you are forced to truncate or "wrap around" the hash because the hash is much larger than the modulus n .
- **Impact:** This creates "collisions." Many different messages could result in the same `hash % n` value, allowing an attacker to swap your message for a malicious one without breaking the signature.
- **Standard:** The modulus n must always be mathematically larger than the hash value being signed.

2. Security of SHA-256

- **Strength:** SHA-256 is a current industry standard. It is "one-way" (you can't turn the hash back into the message) and "collision-resistant."

- **Benefit:** Even changing a single capital letter to lowercase (as seen in your "Hello rsa" test) changes the hash entirely, causing the verification to fail.

3. Key Directionality

- **Analysis:** In encryption, you use the Public Key to hide data. In digital signatures, you use the **Private Key** to sign data.
- **Logic:** Since only you have your private key, only you could have produced that specific signature. Since everyone has your public key, everyone can verify it.

4. Small Modulus n

- **The Risk:** Just like the previous task, $n = 3233$ is trivial to factor. An attacker can find p and q in milliseconds, calculate d , and then forge your signature on any document they want (e.g., a fake bank transfer).

5. Lack of PSS Padding

- **The Issue:** This is "Full Domain Hash" signing without padding.
- **The Risk:** Mathematical attacks (like Existential Forgery) can allow attackers to create valid-looking signatures by manipulating the math if no random padding (like **RSA-PSS**) is used.